5. BÚSQUEDA LOCAL
(PARTE 1)

IA 3.2 - Programación III

1° C - 2026

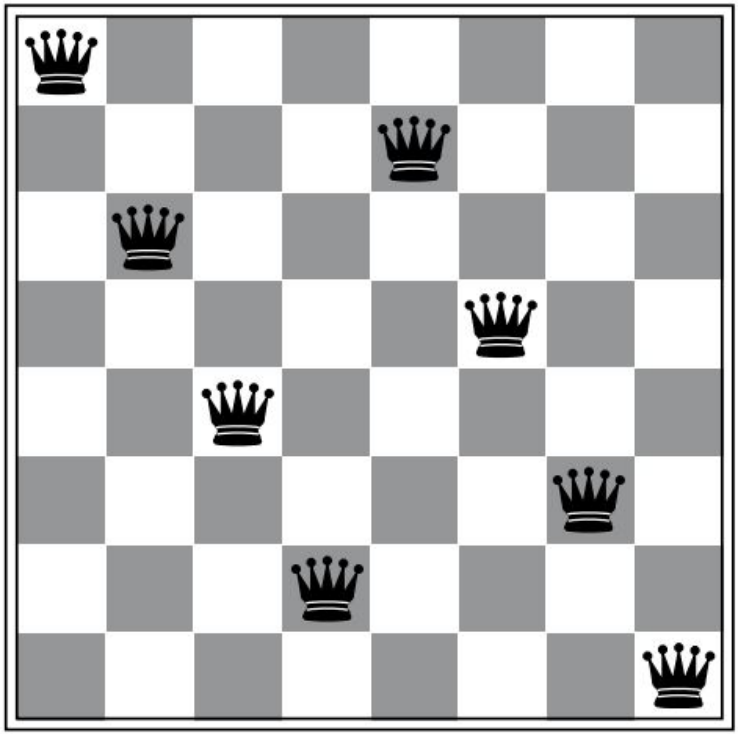
Dr. Mauro Lucci

🔙 Repaso

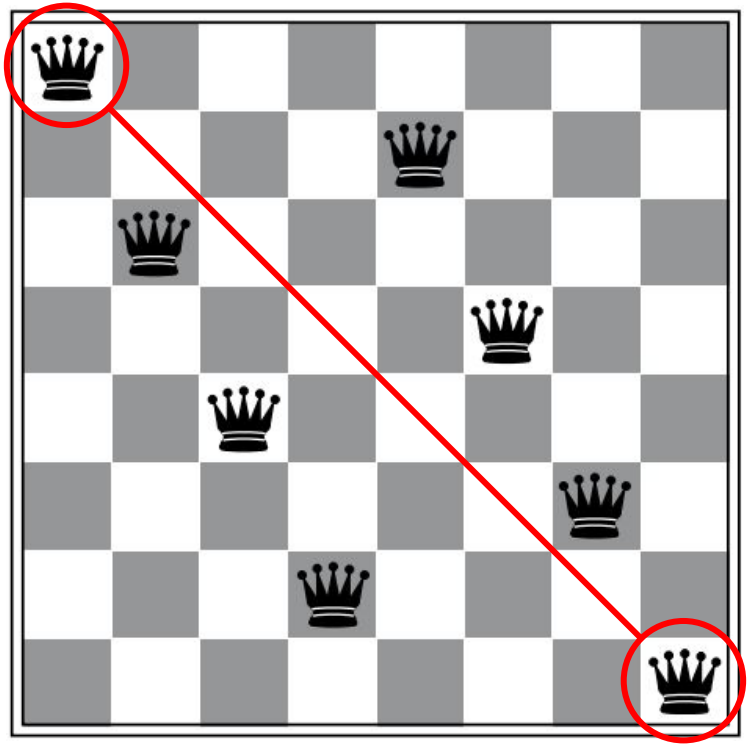
- Los algoritmos de búsqueda que vimos exploran **sistemáticamente** caminos en el espacio de estados.
- Uso **intensivo** de la memoria:
  - Frontera con los nodos que deben seguir explorando.
  - Ancestros para retornar una solución.

Motivación

Problema de las 8 Reinas

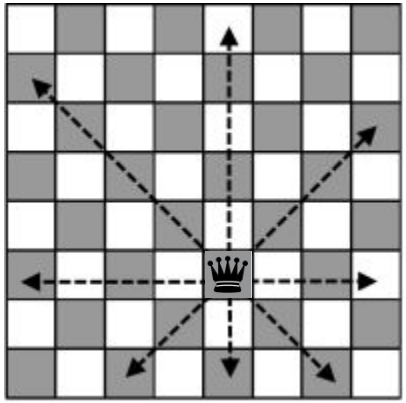


Problema de las 8 Reinas



Descripción

Objetivo. Ubicar 8 reinas en un tablero de ajedrez de forma tal que ninguna reina ataque a otra.



Reglas. Cada casilla puede contener a lo sumo una reina.

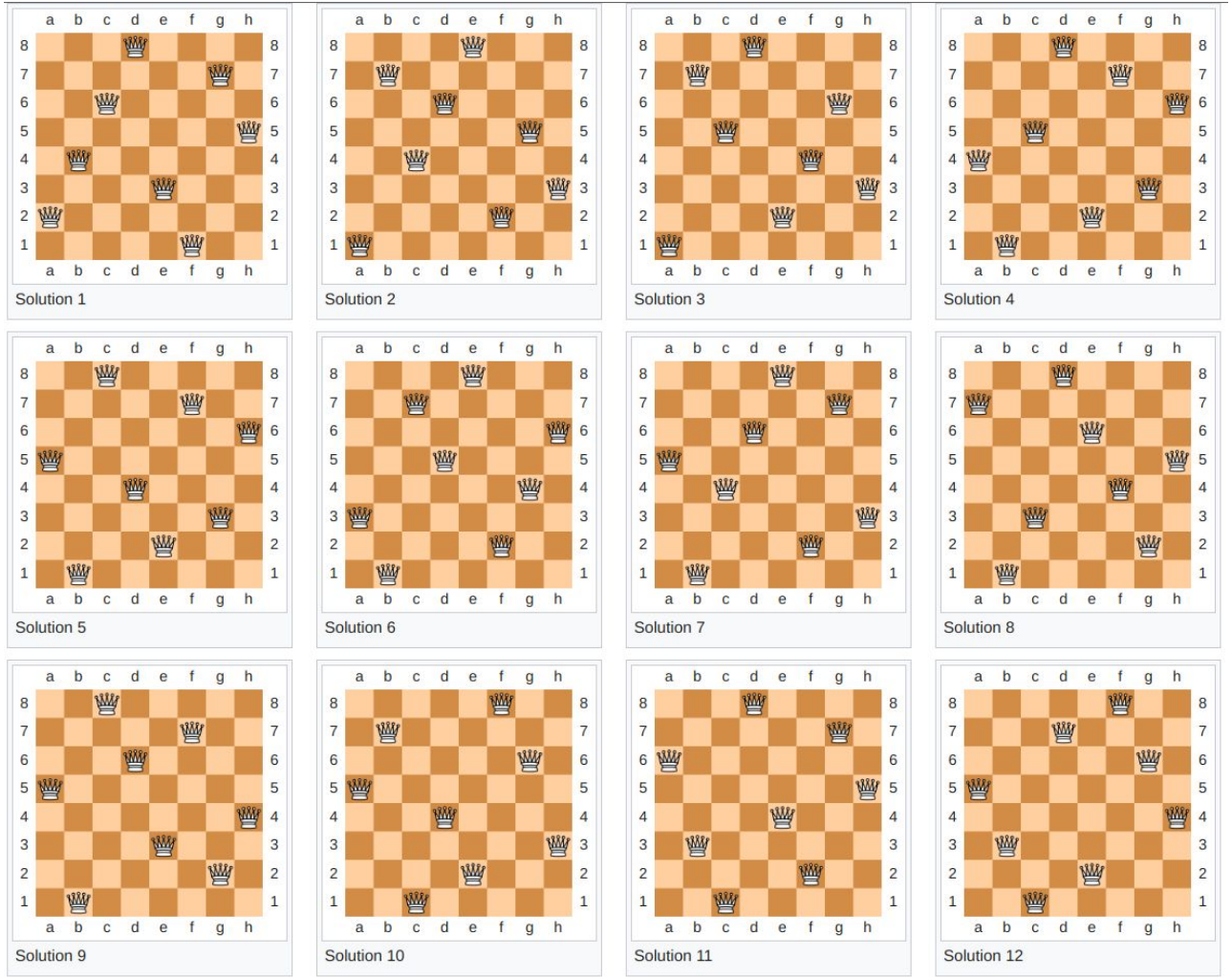
📝 Desafío

¿Es posible resolver el problema?

Respuesta

En total, hay **92** soluciones.

Sin contar soluciones simétricas, hay **12** soluciones.



Formulación I

En esta formulación, un **estado** es una matriz M de 8x8 donde:

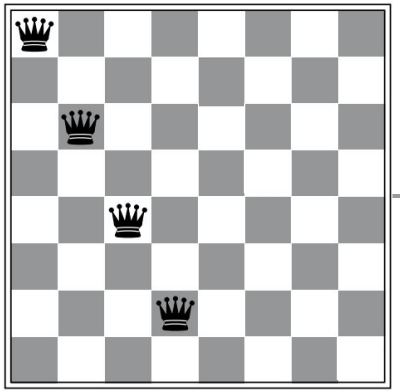
- $M[i][j] = 1$ si hay reina en la fila i y columna j.
- $M[i][j] = 0$ en caso contrario.

tal que $(\Sigma_{ij}\ M[i][j]) \leq 8 \rightarrow$ A lo sumo 8 reinas en el tablero.

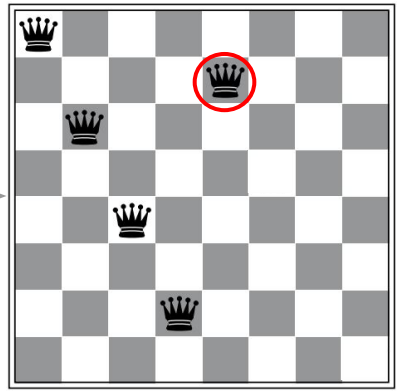
Una **acción** es un par $(i,j) \in \{1,\ldots,8\}^2$ que representa que se coloca una reina en la fila i y columna j.

Formulación I

📖 **Ejemplo.**



Agregar una reina en la fila 2 y columna 5.



1	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	1	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	1	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	0
0	0	0	0	0	0	0	0

(2,5)

1	0	0	0	0	0	0	0
0	0	0	0	**1**	0	0	0
0	1	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	1	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	0
0	0	0	0	0	0	0	0

¿Qué inconvenientes podría tener esta formulación? 🤔

Tamaño del espacio de estados

El número total de estados es **enorme**.

$$1 + 64 + 64.63 + \ldots + 64.63.\ldots.57 \approx \mathbf{1{,}8 \times 10^{14}}$$

Número de matrices con 0 reinas.

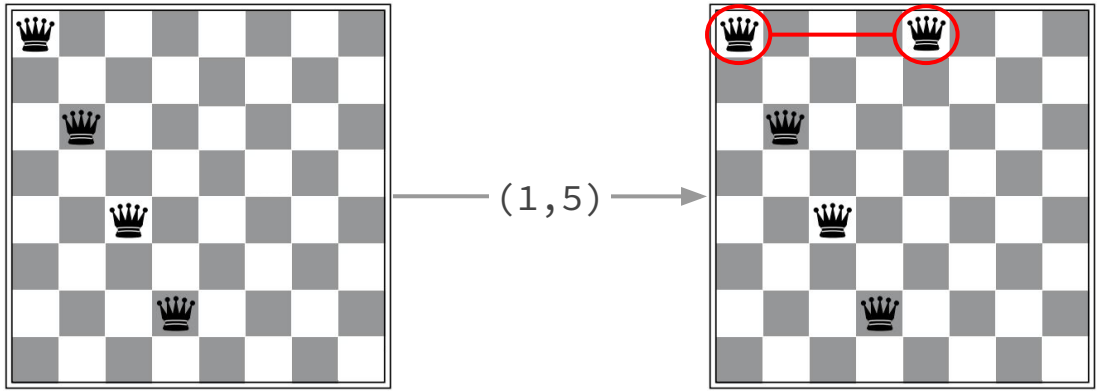
Número de matrices con 1 reina.

Número de matrices con 2 reinas.

Número de matrices con 8 reinas.

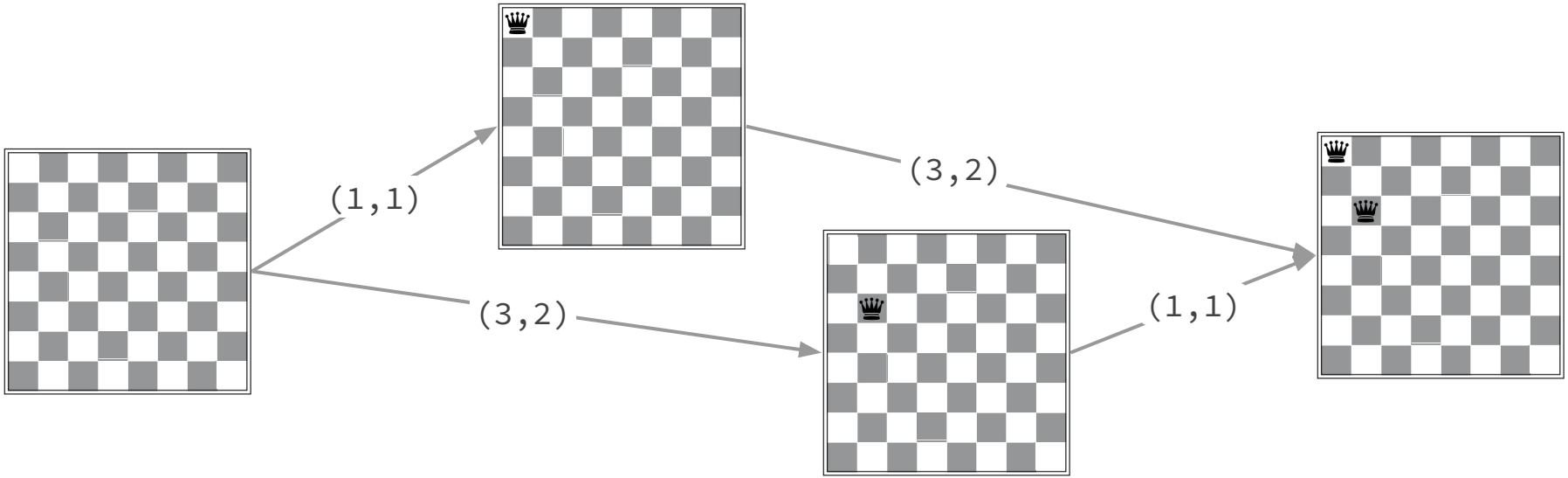
Reinas en conflicto

Las acciones permiten agregar reinas en casillas atacadas.



Caminos redundantes

Las acciones no tienen en cuenta la **simetría** de las acciones.



📝 Ejercicio

Terminar de formular el problema de las 8 reinas como problema de búsqueda, usando la representación matricial.

Respuesta

- **Estado inicial.** $M[i][j] = 0$ para toda fila i y columna j.
- **Acciones.** Dada una matriz M,

$$\text{ACCIONES}(M) = \begin{cases} \{\} & \text{si } (\Sigma_{ij}\ M[i][j]) = 8 \\ \{(i,j) \in \{1,\ldots,8\}^2 : M[i][j] = 0\} & \text{en caso contrario} \end{cases}$$

- **Modelo de transiciones.** Dada una matriz M y una acción (i,j),

  RESULTADO$(M,(i,j)) = M'$, donde:

$$M'[i'][j'] = \begin{cases} 1 & \text{si } i' = i \text{ y } j' = j. \\ M[i'][j'] & \text{en caso contrario.} \end{cases}$$

Más adelante veremos cómo escribir esto formalmente…

- **Test objetivo.** Dada una matriz M, ¿$(\Sigma_{ij}\ M[i][j]) = 8$ y <u>las reinas no se atacan</u>?
- **Costo de camino.** Es la suma de los costos individuales de las acciones.

  Dada una matriz M y una acción (i,j), COSTO-INDIVIDUAL$(M,(i,j)) = 1$.

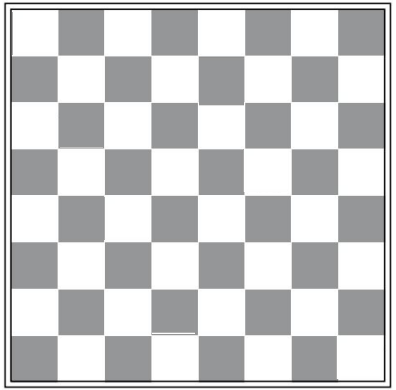
Formulación II

En esta nueva formulación, un **estado** es:

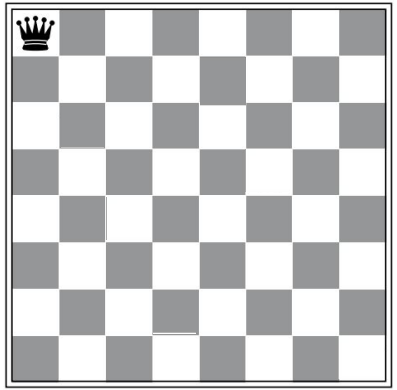
- La tupla vacía (), que representa que no hay reinas, o
- Para $n \in \{1,\ldots,8\}$, una n-tupla $(f_1, f_2, \ldots, f_n)$ donde cada f_i es un número **diferente** del **1** al **8** y representa la fila donde se ubica la reina de la columna i.

Formulación II

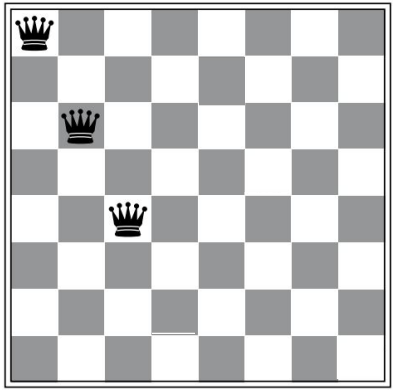
📖 **Ejemplos.**



()



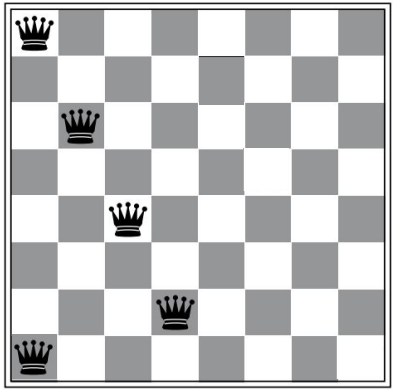
(1)



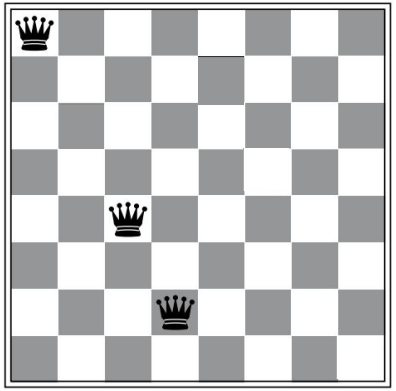
(1,3,5)

Formulación II

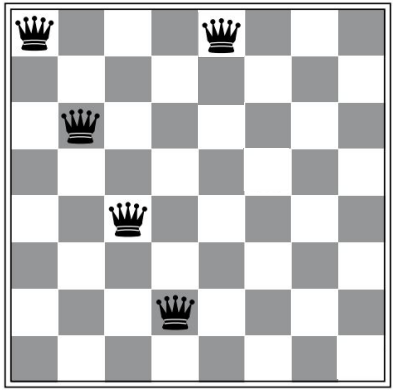
📖 **Ejemplos.** Ahora, **no son estados:**



❌ Hay dos reinas en la columna 1.



❌ No hay reinas en la columna 2, pero hay reinas en las columnas siguientes.



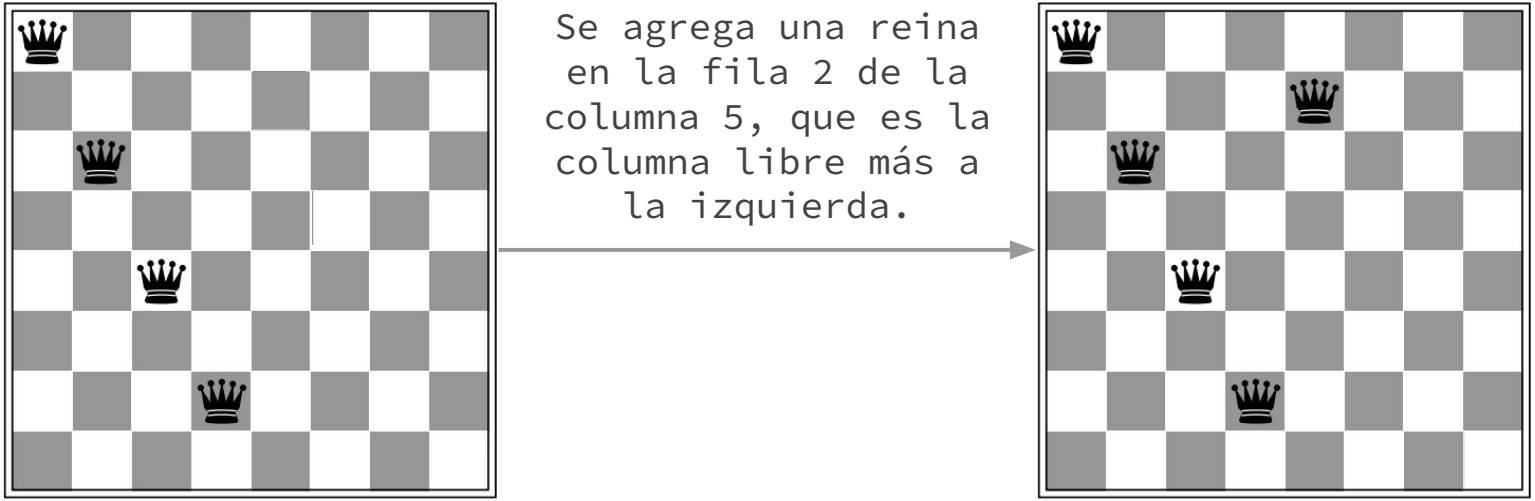
❌ Hay dos reinas en la fila 1.

Formulación II

Una **acción** es un número $m \in \{1,\ldots,8\}$ que representa que se agrega una reina en la fila m de la columna libre más a la izquierda.

Formulación II

📖 **Ejemplos.**



📝 Ejercicio

Terminar de formular el problema de las 8 reinas como problema de búsqueda, usando la representación de tuplas.

Respuesta

- **Estado inicial.** ().
- **Acciones.** Dado un estado s,

$$\text{ACCIONES}(s) = \begin{cases} \{1,\ldots,8\} & \text{si } s = () \\ \{1,\ldots,8\} \,/\, \{f_1,\ldots,f_n\} & \text{si } s = (f_1,\ldots,f_n) \text{ para algún } n \in \{1,\ldots,7\} \\ \{\} & \text{en caso contrario.} \end{cases}$$

- **Modelo de transiciones.** Dado un estado s y una acción m,

$$\text{RESULTADO}(s,m) = \begin{cases} (m) & \text{si } s = (). \\ (f_1,\ldots,f_n,m) & \text{si } s = (f_1,\ldots,f_n) \text{ para algún } n \in \{1,\ldots,7\}. \end{cases}$$

- **Test objetivo.** Dado un estado $(f_1,\ldots,f_n)$, ¿n = 8 y <u>las reinas no se atacan diagonalmente</u>.
- **Costo de camino.** Es la suma de los costos individuales de las acciones. Dado un estado s y una acción m, COSTO-INDIVIDUAL(s,m) = 1.

Más adelante veremos cómo escribir esto formalmente…

Tamaño del espacio de estados

El número de estados es:

$$1 + 8 + 8.7 + \ldots + 8.7.6.5.4.3.2.1 = \mathbf{109601}$$

Tupla vacía.

Número de tuplas con 1 número del 1 al 8.

Número de tuplas con 2 números distintos del 1 al 8.

Número de tuplas con 8 números distintos del 1 al 8.

¡Muchos menos que en la primera formulación! Si además no permitimos estados donde las reinas se ataquen diagonalmente, baja a **2057** (la llamaremos formulación II*).

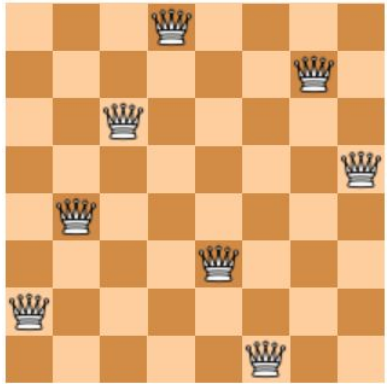
Formulación I vs. II*

- Para 8 reinas,
  - Formulación I: $1{,}8 \times 10^{14}$ estados.
  - Formulación II*: 2057 estados.
- Para 100 reinas,
  - Formulación I: 10^{400} estados.
  - Formulación II*: 10^{52} estados.

Sería casi imposible resolver el problema de las 100 reinas con los algoritmos de búsqueda vistos hasta ahora… El número de nodos en memoria sería **demasiado grande…**
¿Qué hacemos? 🤔

Deducción de soluciones

En muchos problemas de búsqueda, si conocemos un estado objetivo, es trivial encontrar una solución (un camino en el espacio de estados desde el estado inicial a él).



📖 **Ejemplo**. Para el estado de la izquierda, y considerando la formulación II, se tiene la siguiente secuencia de acciones:

7 → 5 → 3 → 1 → 6 → 8 → 2 → 4

Es decir, primero se coloca una reina en la fila 7 de la columna 1, luego una reina en la fila 5 de la columna 2, y así sucesivamente…

💡 Nuevos algoritmos de búsqueda

- No almacenar el **padre**.
- No almacenar a los **ancestros**.

  (Pero la ganancia de memoria puede no ser suficiente, ya hemos hablando del tamaño de la frontera)

- No almacenar la **frontera** 🤯.

Búsqueda local

Local search

Una **búsqueda local** mantiene unos **pocos** estados en memoria. En cada iteración, **actualiza** el estado actual por alguno de sus sucesores (a.k.a. *se mueve a un sucesor*), **eligiéndolo** con algún criterio y **deteniéndose** con algún criterio.

Hay una **amplia** familia de algoritmos que se pueden clasificar como búsqueda local.

Aplicación. Problemas donde lo importante es encontrar un estado objetivo (y no el camino al mismo).

💻 Algoritmo general de Búsqueda Local

```
function LOCAL-SEARCH(problema) return estado
    s ← problema.estado-inicial
    while no se cumple criterio de parada do:
        S’ ← {problema.resultado(s,a) for
                a ϵ problema.acciones(s)}
        elegir un buen sucesor s’ ϵ S’
        s ← s’
    return s
```

Conjunto de estados sucesores, definido por **comprensión**.

💻 Algoritmo general de Búsqueda Local

```
function LOCAL-SEARCH(problema) return estado
    s ← problema.estado-inicial
    while no se cumple criterio de parada do:
        S’ ← {problema.resultado(s,a) for
                a ϵ problema.acciones(s)}
        elegir un buen sucesor s’ ϵ S’
        s ← s’
    return s
```

Observación 1. Para dar una implementación concreta de este algoritmo, hay que decidir una **estrategia de elección** de sucesor y un **criterio de parada.**

Observación 2. A diferencia de la búsqueda de árbol, no hay nodos, ni frontera.

¿Qué inconvenientes podrían tener las implementaciones de este algoritmo? 🤔

Características generales

- ✅ Muy bajo consumo de **memoria**.
- ✅ Puede encontrar soluciones **razonablemente buenas** en espacios de estados muy grandes (o infinitos).
- ❌ Se **atasca** en estados terminales (sin sucesores).
- ❌ Puede **ciclar**.
- ❌ No garantiza **completitud** ni **optimalidad**.

Formulación incremental

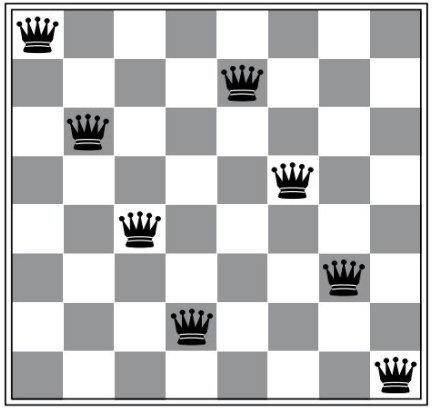
En una **formulación incremental** las acciones agregan nuevas componentes al estado. Por ejemplo, las formulaciones I y II que vimos para el problema de las reinas.

Las formulaciones incrementales **no son compatibles** con **búsqueda local**, dado que **es imposible progresar cuando se llega a un estado terminal que no es objetivo.**

📖 Ejemplo – 8 Reinas

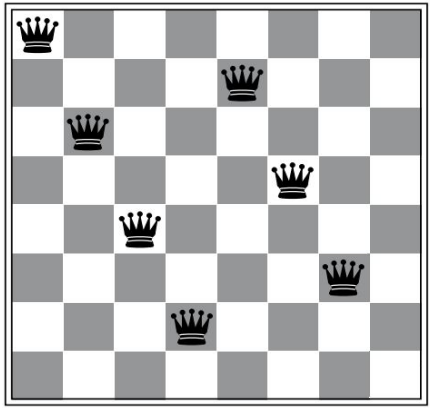
Una búsqueda local se atasca en los siguientes estados no-objetivos:

En la formulación I:



No es posible agregar más reinas, pues ya están las 8.

En la formulación II*:



Falta agregar 1 reina pero no quedan casillas libres no atacadas.

Formulación de estados completa

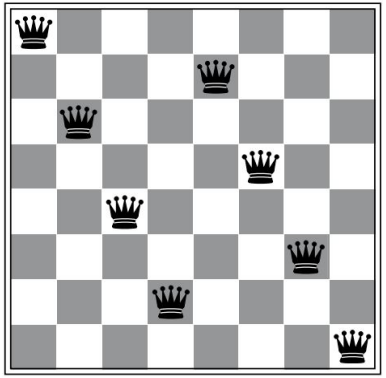
Con búsqueda local, se usan típicamente **formulaciones de estados completa**, donde cada estado ya tiene ubicadas todas las componentes y siempre es posible aplicar acciones que muevan o intercambien componentes ya ubicadas.

📖 Ejemplo – 8 Reinas

En esta formulación de estados completa, un **estado** es una 8-tupla $(f_1, f_2, \ldots, f_8)$ donde, para cada $i \in \{1,\ldots,8\}$, $f_i \in \{1,\ldots,8\}$ representa la fila donde se ubica la reina de la columna i. Es decir, **todas las reinas ya están ubicadas en el tablero**.

El número total de estados es

$$8^8 = \mathbf{16.777.216}.$$



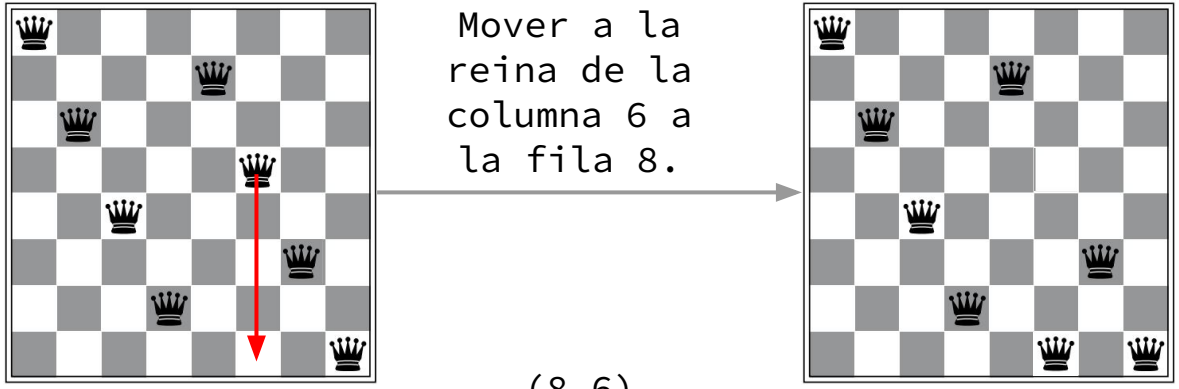
Representación:
(1,3,5,7,2,4,6,8)

Observación. En esta formulación, las reinas se pueden atacar horizontalmente y diagonalmente.

📖 Ejemplo – 8 Reinas

Una **acción** es un par $(i,j) \in \{1,\ldots,8\}^2$ que representa mover la reina de la columna j a la fila i de la misma columna.

Cada estado tiene 7x8 = **56** sucesores y **no hay estados terminales**.



(1,3,5,7,2,4,6,8) → (1,3,5,7,2,**8**,6,8)

📖 Ejemplo – 8 Reinas

- **Estado inicial.** Cualquier estado, por ej. todas las reinas en la fila 1:

$$(1,1,1,1,1,1,1,1).$$

- **Acciones.** Dado un estado $(f_1,\ldots,f_8)$,

$$\text{ACCIONES}(f_1,\ldots,f_8) = \{(i,j) \in \{1,\ldots,8\}^2 : i \neq f_j\},$$

  es decir, mover cualquier reina a una fila diferente de la columna en la que se encuentra.

- **Modelo de transiciones.** Dados un estado $(f_1,\ldots,f_8)$ y una acción (i,j),

$$\text{RESULTADO}((f_1,\ldots,f_{j-1},f_j,f_{j+1},\ldots,f_8),(i,j)) = (f_1,\ldots,f_{j-1},\mathbf{i},f_{j+1},\ldots,f_8).$$

- **Estado objetivo.** Dado un estado $(f_1,\ldots,f_8)$, ¿las reinas se atacan?
- **Costo de camino.** ¿?

Costos de camino

En una búsqueda local, los costos de camino son **irrelevantes**, dado que **se sigue un único camino y no se lleva ningún registro de las alternativas que quedaron sin explorar**.

Entonces… ¿cómo elige buenos sucesores? 🤔

Función objetivo

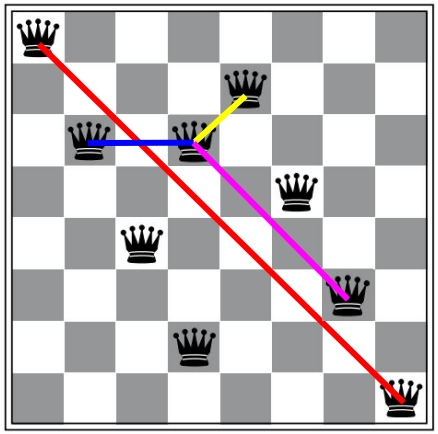
- Los algoritmos de búsqueda local, se guían por medio de una **función objetivo**.
- Una función objetivo f, asigna a cada estado s, un valor numérico $f(s)$, conocido como **valor objetivo** (de s por f).
- Una función objetivo tiene que alcanzar su **máximo valor** (o mínimo) en los estados objetivos del problema.
- Una búsqueda local elige los sucesores en pos de **maximizar** (o minimizar) una cierta función objetivo.

📖 Ejemplo – 8 Reinas

Función objetivo.

f: número de pares de reinas atacadas.

s =



$f(s) = 4$

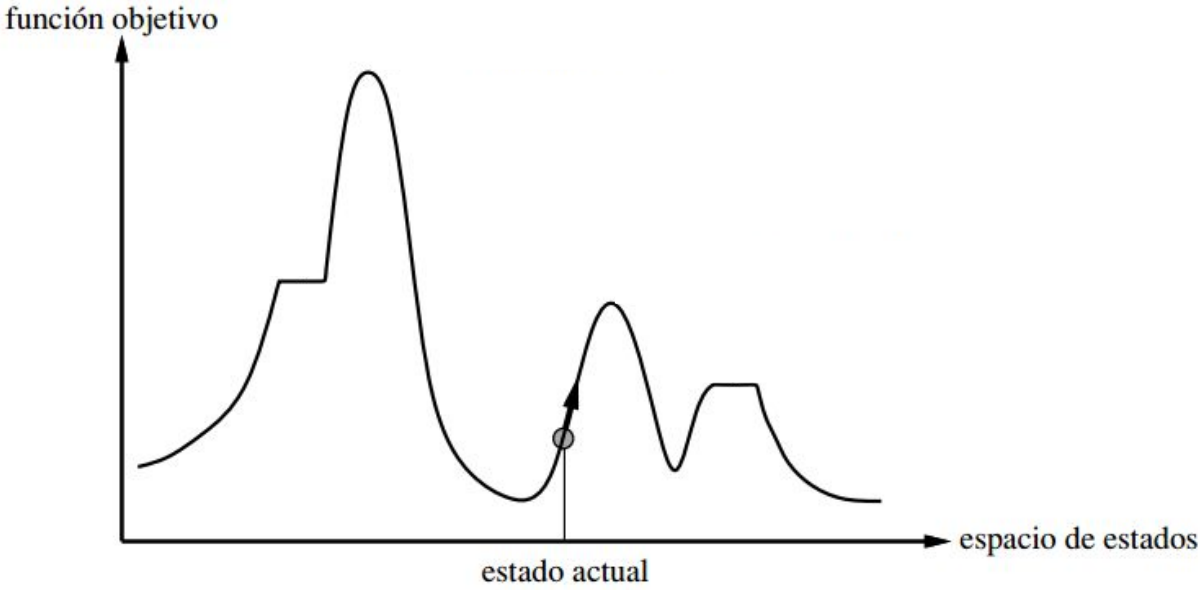
👉 **Se busca minimizar f.**

En particular, para todo estado objetivo s', $f(s') = 0$.

📌 **Nota.** Minimizar f es equivalente a maximizar $-f$.

Paisaje del espacio de estados

En un problema donde se debe maximizar la función objetivo, la búsqueda local explora el siguiente **paisaje**.



📝 Ejercicio

Proponer una formulación de estados completa y una función objetivo para el problema del viajante (TSP).

Notación.

$\{c_1,\ldots,c_n\}$: conjunto de ciudades, suponer que c_1 es la inicial.

$\mathrm{dist}(c_i,c_j)$: distancia en km entre todo par de ciudades c_i y c_j.

📝 Respuesta

En esta formulación, un **estado** es una (n+1)-tupla

$$\left(c_{j_1}, c_{j_2} \ldots, c_{j_n}, c_{j_{n+1}}\right)$$

tal que:

- $c_{j_1} = c_1$ (c_1 es la ciudad inicial).
- $c_{j_{n+1}} = c_1$ (c_1 es la ciudad final).
- para todo $i \in \{2,\ldots,n\}$, c_{j_i} es una ciudad diferente de $\{c_2,\ldots,c_n\}$.

el cual representa el orden en que se recorren las ciudades: se inicia en $c_{j_1} = c_1$, luego se visita c_{j_2}, y así sucesivamente, hasta visitar a c_{j_n} y finalmente regresar a $c_{j_{n+1}} = c_1$.

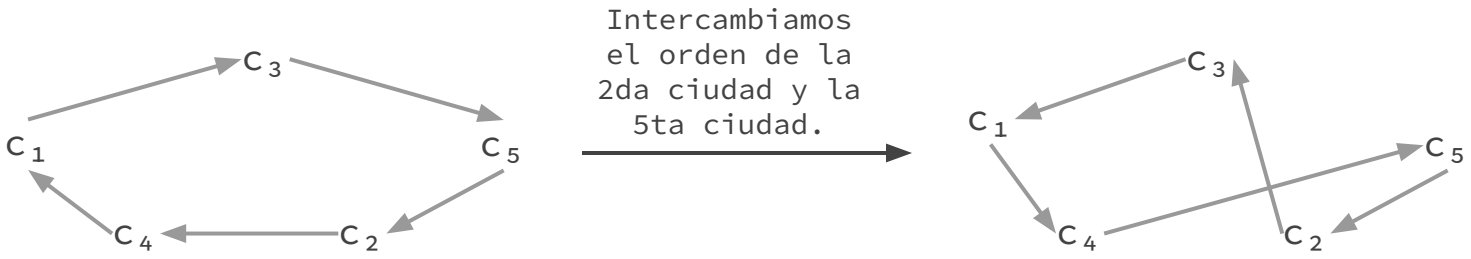
El número total de estados es (n-1).(n-2).….2.1 = **(n-1)!**

📝 Respuesta

En la literatura existen otras propuestas de acciones.

Una **acción** es un par $(i,k) \in \{2,\ldots,n\}^2$, con $i<k$, que representa intercambiar de lugar la i-ésima ciudad del tour por la j-ésima ciudad del tour.

Por ejemplo, si $n = 5$:



Representación: $(c_1, c_3, c_5, c_2, c_4, c_1) \xrightarrow{(2,5)} (c_1, c_4, c_5, c_2, c_3, c_1)$

📝 Respuesta

- **Estado inicial.** Cualquier estado, por ejemplo:

$$(c_1, c_2, \ldots, c_n, c_1)$$

- **Acciones.** Dado un estado $(c_{j_1}, \ldots, c_{j_{n+1}})$,

  ACCIONES $(c_{j_1}, \ldots, c_{j_{n+1}})$ = {(*i*,*k*) ∈ {2,…,*n*}²: *i* < *k*}

- **Modelo de transiciones.** Dado un estado $(c_{j_1}, \ldots, c_{j_i}, \ldots, c_{j_k}, \ldots, c_{j_{n+1}})$ y una acción (*i*,*k*),

  RESULTADO($(c_{j_1}, \ldots, c_{j_i}, \ldots, c_{j_k}, \ldots, c_{j_{n+1}})$,(*i*,*k*)) = $(c_{j_1}, \ldots, c_{j_k}, \ldots, c_{j_i}, \ldots, c_{j_{n+1}})$

- **Test objetivo.** Todo estado es objetivo.

- **Función objetivo.** Dado un estado $(c_{j_1}, \ldots, c_{j_{n+1}})$, queremos minimizar

$$f(c_{j_1}, \ldots, c_{j_{n+1}}) = \sum_{i=1}^{n} dist(c_{j_i}, c_{j_{i+1}})$$

Estrategias de elección de sucesores

- Con una formulación de estados completa, la búsqueda local **siempre** tiene sucesores donde **moverse**.
- Mediante la función objetivo puede identificar a los sucesores más **prometedores**.
- Entonces… ¿cuál elige? 🤔

A continuación, veremos diferentes **estrategias**.

Búsqueda de ascensión de colinas

Hill-climbing search

La **búsqueda de ascensión de colinas** es la búsqueda local más simple.

Supongamos que queremos maximizar la función objetivo. En cada iteración se mueve al sucesor con **mayor** valor objetivo, siempre que mejore el valor objetivo actual, hasta **atascarse**.

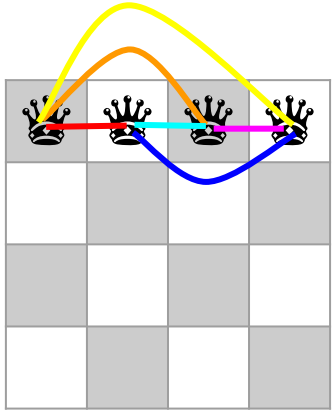
En el paisaje del espacio de estados, se mueve hacia **arriba** hasta que ya no sea posible.

También es llamada búsqueda local **avara (greedy)**, ya que no mira más allá de los sucesores inmediatos.

📖 Ejemplo – 4 Reinas

Veamos un ejemplo del algoritmo de ascensión de colinas para resolver el problema de las 4 reinas. Usamos su formulación de estados completa y **minimizamos** la función f que cuenta el número de pares de reinas que se atacan.

📖 Ejemplo – 4 Reinas



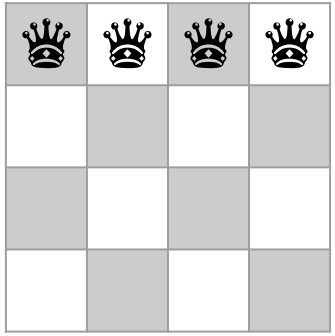
f(1,1,1,1) = 6

Supongamos que el estado inicial es (1,1,1,1), es decir, todas las reinas están en la fila 1.

El valor objetivo del estado inicial es 6 (hay 6 pares de reinas en ataque).

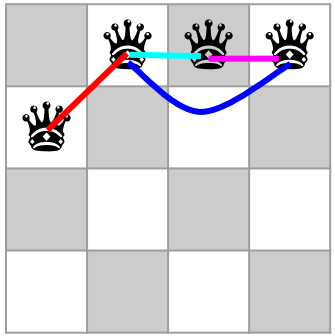
Para elegir dónde movernos, tenemos que calcular el valor objetivo de todos los sucesores…

📖 Ejemplo – 4 Reinas



f(1,1,1,1) = 6

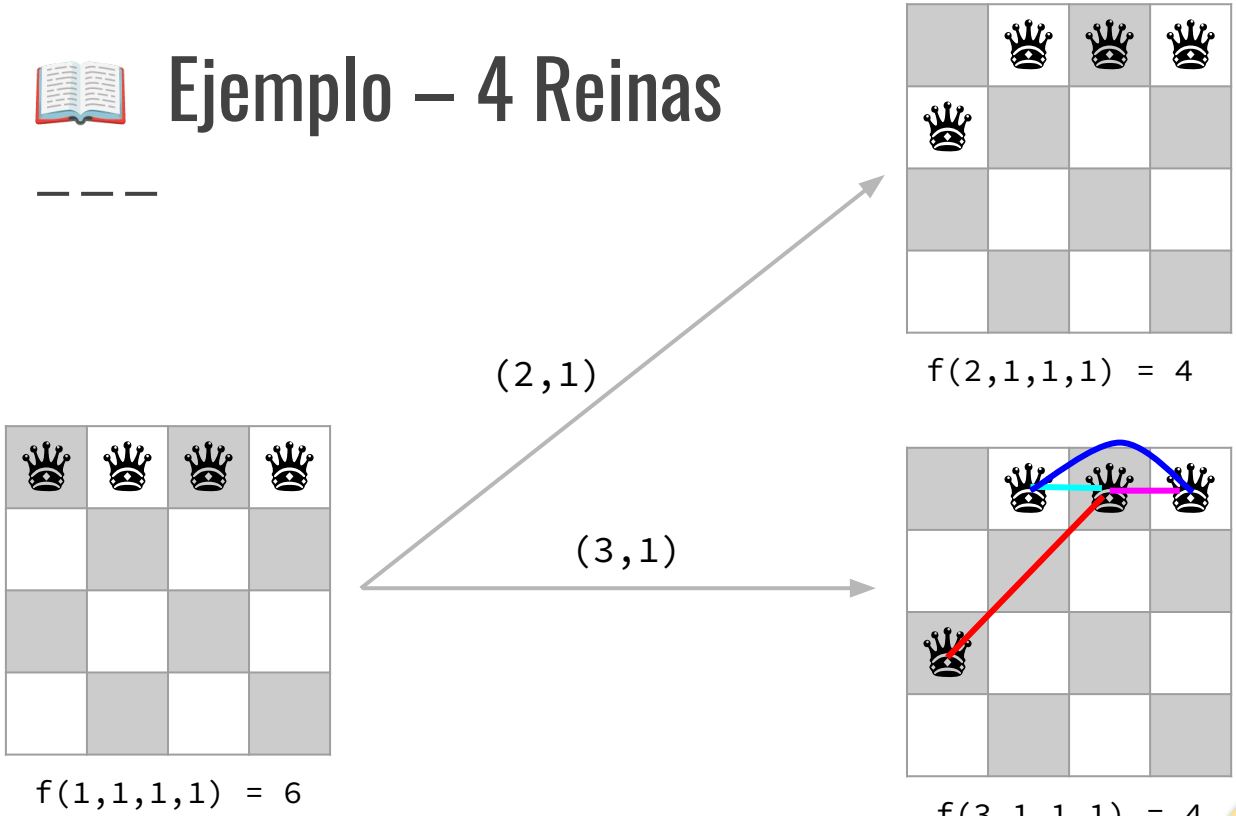
(2,1)



f(2,1,1,1) = 4

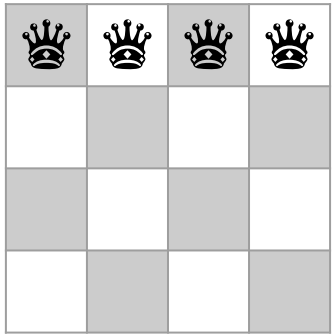
Moviendo la reina de la columna 1 a la fila 2, el valor objetivo decrece a 4

📖 Ejemplo – 4 Reinas



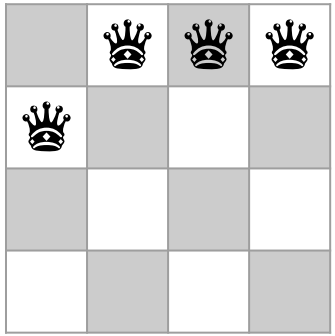
Moviendo la reina de la columna 1 a la fila 3, el valor objetivo decrece a 4

📖 Ejemplo – 4 Reinas



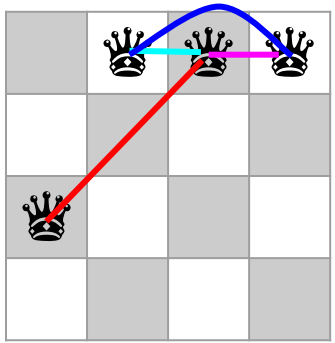
f(1,1,1,1) = 6

(2,1)



f(2,1,1,1) = 4

(3,1)



f(3,1,1,1) = 4

(4,1)

Continúa…

Debemos continuar con el resto de sus sucesores…

📖 Ejemplo – 4 Reinas

🙁	🙁	🙁	🙁
4			
4			
…			

f(1,1,1,1) = 6

Para ahorrar espacio, vamos a escribir sobre el estado actual el valor objetivo de cada uno de sus sucesores.

El número de la fila *i* y columna *j* representará el valor objetivo del estado que se obtiene de aplicar la acción (*i*,*j*) al estado actual.

Adelantemos algunos pasos…

📖 Ejemplo – 4 Reinas

¿A qué sucesor nos tenemos que mover? 🤔

🥱	🥱	🥱	🥱
4	5	5	4
4	4	4	4
4	3	3	4

`f(1,1,1,1) = 6`

📖 Ejemplo – 4 Reinas

🙁	🙁	🙁	🙁
4	5	5	4
4	4	4	4
4	**3**	**3**	4

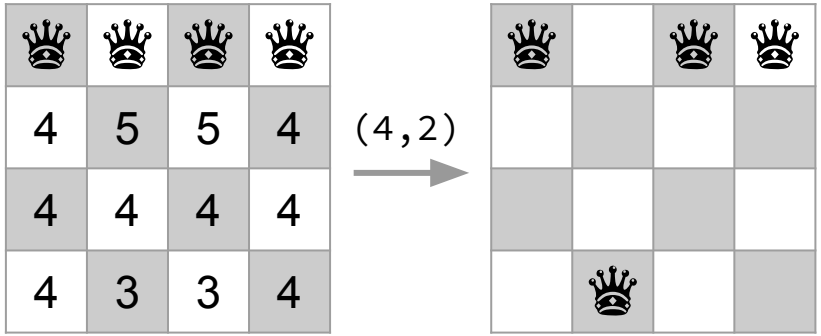
f(1,1,1,1) = 6

¿A qué sucesor nos tenemos que mover? 🤔

El número más chico es 3 y se encuentra en las casillas (4,2) y (4,3). Esto quiere decir, que los estados sucesores con menor valor objetivo se obtiene moviendo la reina de la columna 2 o 3 a la fila 4. Además, dichos valores objetivos son menores al actual (3 < 6).

La estrategia a priori no tiene una regla de desempate, por lo tanto cualquiera de ellos puede ser elegido. Por ejemplo, apliquemos la acción (4,2)

📖 Ejemplo – 4 Reinas



`f(1,1,1,1) = 6`

`f(1,4,1,1) = 3`

📖 Ejemplo – 4 Reinas

🙁	🙁	🙁	🙁
4	5	5	4
4	4	4	4
4	3	3	4

f(1,1,1,1) = 6

(4,2) →

🙁		🙁	🙁
	🙁		

f(1,4,1,1) = 3

Calculemos nuevamente los valores objetivo de los sucesores.

📖 Ejemplo – 4 Reinas

🙁	🙁	🙁	🙁
4	5	5	4
4	4	4	4
4	3	3	4

f(1,1,1,1) = 6

(4,2) →

🙁	6	🙁	🙁
1	5	2	3
3	4	2	1
3	🙁	2	3

f(1,4,1,1) = 3

Observación. Algunos sucesores pueden empeorar el valor objetivo. Por ejemplo, en el estado (1,4,1,1), mover la reina de la columna 2 a la fila 1 o 2, aumenta el número de pares de reinas atacadas.

📖 Ejemplo – 4 Reinas

🙁	🙁	🙁	🙁
4	5	5	4
4	4	4	4
4	3	3	4

f(1,1,1,1) = 6

(4,2) →

🙁	6	🙁	🙁
1	5	2	3
3	4	2	1
3	🙁	2	3

f(1,4,1,1) = 3

¿A qué sucesor nos tenemos que mover ahora? 🤔

📖 Ejemplo – 4 Reinas

🙁	🙁	🙁	🙁
4	5	5	4
4	4	4	4
4	3	3	4

f(1,1,1,1) = 6

(4,2) →

🙁	6	🙁	🙁
1	5	2	3
3	4	2	1
3	🙁	2	3

f(1,4,1,1) = 3

¿A qué sucesor nos tenemos que mover ahora? 🤔

Podemos mover indistintamente la reina de la columna 1 a la fila 2 o la de la columna 4 a la fila 3, ya que ambas acciones son las que obtienen los sucesores con menor valor objetivo y mejoran el valor objetivo actual (1 < 3). Por ejemplo, apliquemos la acción (2,1).

📖 Ejemplo – 4 Reinas

🤴	🤴	🤴	🤴
4	5	5	4
4	4	4	4
4	3	3	4

f(1,1,1,1) = 6

(4,2) →

🤴	6	🤴	🤴
1	5	2	3
3	4	2	1
3	🤴	2	3

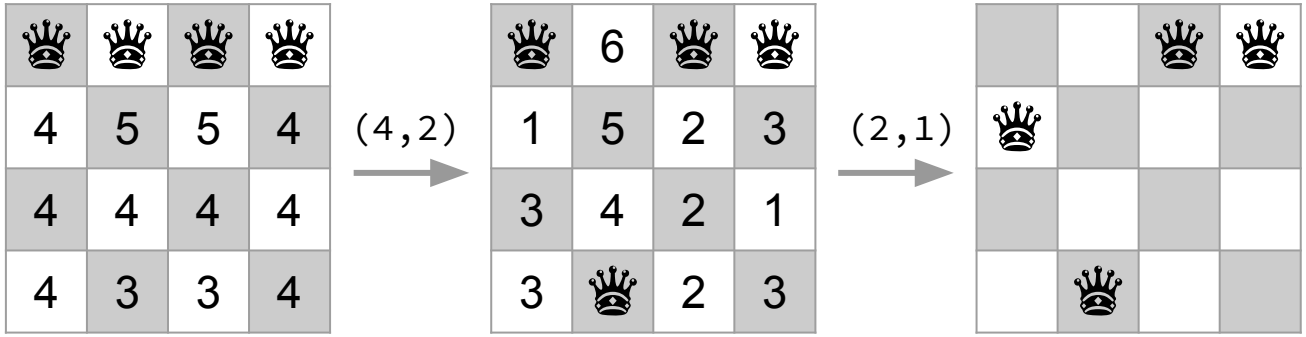
f(1,4,1,1) = 3

(2,1) →

		🤴	🤴
🤴			
	🤴		

f(2,4,1,1) = 1

📖 Ejemplo – 4 Reinas



f(1,1,1,1) = 6

f(1,4,1,1) = 3

f(2,4,1,1) = 1

Calculemos nuevamente los valores objetivo de los sucesores.

📖 Ejemplo – 4 Reinas

👸	👸	👸	👸
4	5	5	4
4	4	4	4
4	3	3	4

f(1,1,1,1) = 6

(4,2) →

👸	6	👸	👸
1	5	2	3
3	4	2	1
3	👸	2	3

f(1,4,1,1) = 3

(2,1) →

3	4	👸	👸
👸	3	2	3
3	3	1	0
3	👸	2	1

f(2,4,1,1) = 1

¿A qué sucesor nos tenemos que mover? 🤔

📖 Ejemplo – 4 Reinas

👸	👸	👸	👸
4	5	5	4
4	4	4	4
4	3	3	4

f(1,1,1,1) = 6

(4,2) →

👸	6	👸	👸
1	5	2	3
3	4	2	1
3	👸	2	3

f(1,4,1,1) = 3

(2,1) →

3	4	👸	👸
👸	3	2	3
3	3	1	**0**
3	👸	2	1

f(2,4,1,1) = 1

¿A qué sucesor nos tenemos que mover? 🤔

La acción (3,4) es la que genera el sucesor con menor valor objetivo y mejora el actual (0 < 1).

📖 Ejemplo – 4 Reinas

🙁	🙁	🙁	🙁
4	5	5	4
4	4	4	4
4	3	3	4

f(1,1,1,1) = 6

(4,2) →

🙁	6	🙁	🙁
1	5	2	3
3	4	2	1
3	🙁	2	3

f(1,4,1,1) = 3

(2,1) →

3	4	🙁	🙁
🙁	3	2	3
3	3	1	0
3	🙁	2	1

f(2,4,1,1) = 1

→

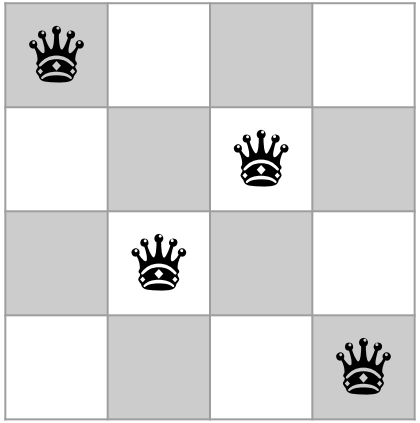
		🙁	
🙁			
			🙁
	🙁		

f(2,4,1,3) = 0

Encontramos un estado objetivo. ¿Ocurre siempre? 🤔

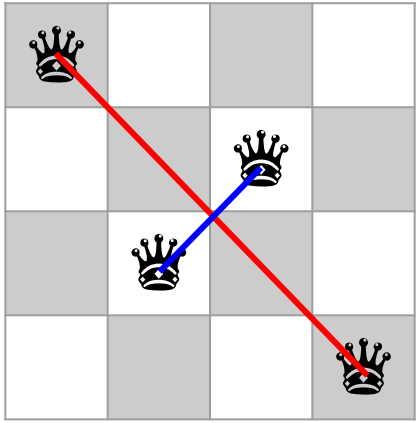
📝 Ejercicio

Aplicar el algoritmo de ascensión de colinas para resolver el problema de las 4 reinas desde el siguiente estado inicial:



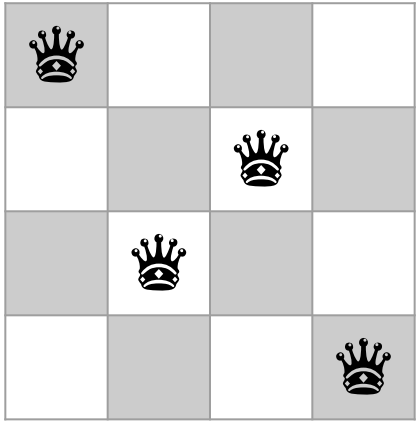
¡¡ Recordar que nunca nos movemos a un sucesor que no mejore el valor actual !!

Respuesta



f(1,3,2,4) = 2

Respuesta



f(1,3,2,4) = 2

Calculemos el valor objetivo de cada uno de sus sucesores…

📝 Respuesta

👸	3	2	4
3	4	👸	2
2	👸	4	3
4	2	3	👸

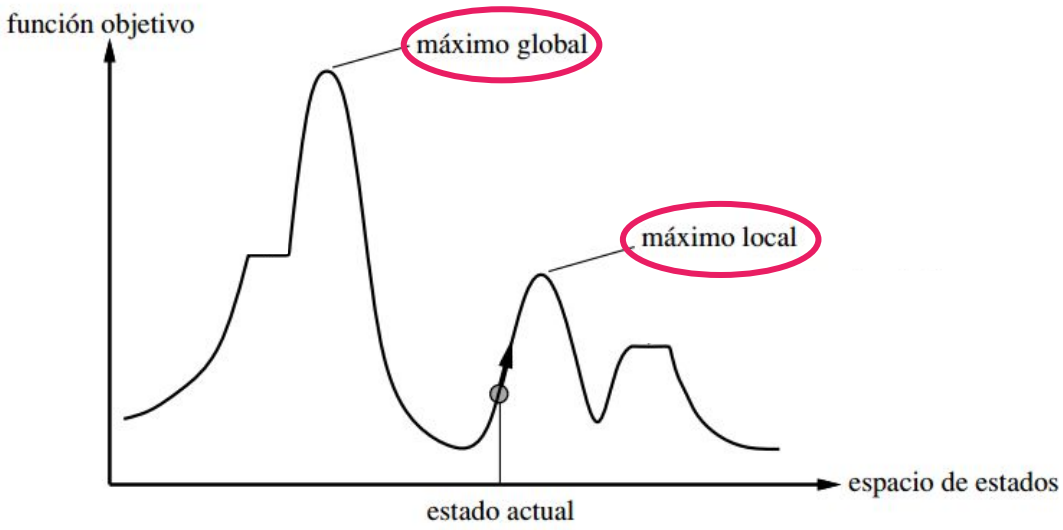
f(1,3,2,4) = 2

Ningún sucesor tiene un valor objetivo que mejore el actual.

La ascensión de colinas queda **atascada** es un estado **no-objetivo.**

¿Por qué ocurre esto? 🤔

Máximos locales y globales

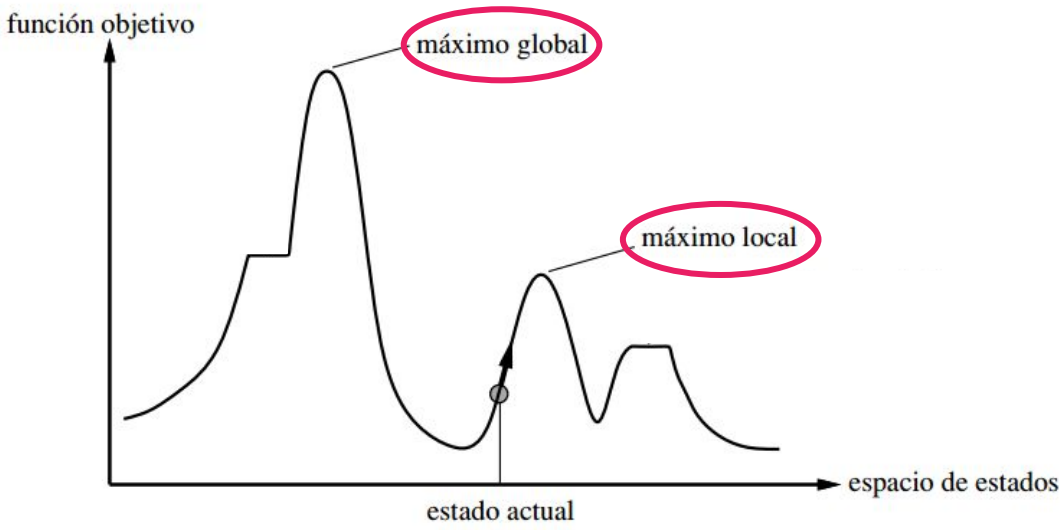


La ascensión de colinas se caracteriza por realizar un **rápido ascenso** por el paisaje del espacio de estados.

Pero una vez que llega a una cima, se **atasca**.

Dicha cima puede ser un **máximo global** o un **máximo local** de la función objetivo.

Máximos locales y globales



Dado un estado *s*, decimos que su valor objetivo *f*(*s*) es un **máximo local** si es mayor o igual al valor objetivo de todos sus sucesores, es decir, si:

f(*s*') ≤ *f*(*s*) para todo sucesor *s*' de *s*.

Y decimos que es un **máximo global** si es un máximo entre todos los máximos locales.

📖 Ejemplo – 4 Reinas

♛	3	2	4
3	4	♛	2
2	♛	4	3
4	2	3	♛

f(1,3,2,4) = 2

Es un mínimo local.

1	3	♛	1
♛	2	2	3
3	2	2	♛
1	♛	3	1

f(2,4,1,3) = 0

Es un mínimo global.

💻 Algoritmo de búsqueda de ascensión de colinas

```
function ASCENSIÓN-COLINAS(problema) return estado
  actual ← problema.estado-inicial
  do
      accion ← MAX-ACCION(problema, actual)
      sucesor ← problema.resultado(actual, accion)
      if (problema.f(sucesor) ≤ problema.f(actual)) then return actual
      actual ← sucesor
```

MAX-ACCION es una función que retorna la acción que conduce al sucesor de *actual* con mayor valor objetivo.

💻 Algoritmo de búsqueda de ascensión de colinas

Una implementación sencilla para MAX-ACCION:

```
1 function MAX-ACCION(problema, s: estado) return accion
2    sucesores ← {a: problema.f(problema.resultado(s,a))
                  for a in problema.acciones(s)}
3    return max(sucesores, key=sucesores.get)
```

sucesores es un diccionario definido por comprensión con pares {acción: valor objetivo}.

max (función de Python) retorna la clave (acción) con el mayor valor.

Usualmente se la define de forma más optimizada, sin generar explícitamente a los estados sucesores ni calcular su valor objetivo desde cero.

Características de ASCENSIÓN-COLINAS

- Muy bajo consumo de **memoria**.
- Se mueve siempre a un sucesor con valor objetivo **estrictamente mayor**.

  ⇒ **Nunca cicla.**

- Se atasca en un **máximo** (local o global).
- En general, no hay garantía de que el estado retornado sea objetivo (**completitud**), ni que el máximo alcanzado sea global (**optimalidad**). Veamos algunos ejemplos…

Performance de ASCENSIÓN-COLINAS

La probabilidad y el tiempo requerido para alcanzar un máximo (local o global) depende del **problema** y de la **función objetivo**.

📖 Ejemplo – 8 Reinas

- Tiene casi **17 millones** de estados (form. de estados completa).
- Empezando la ascensión de colinas desde un estado al azar,
  - Probabilidad de atascarse en un mínimo local: **86%**.
    Promedio de pasos: **3**.
  - Probabilidad de atascarse en un mínimo global: **14%**.
    Promedio de pasos: **4** pasos.
- Es decir, 1 de cada 7 veces, alcanza un mínimo global.
- **¡No está nada mal!** Pero podemos hacerlo mejor…

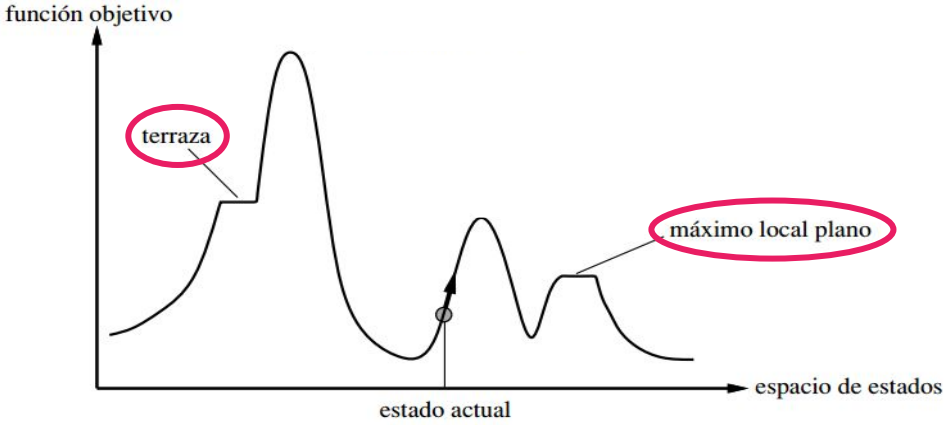
Variantes de la Ascensión de Colinas

A continuación, veremos algunas **variantes** muy conocidas para la Búsqueda de Ascensión de Colinas.

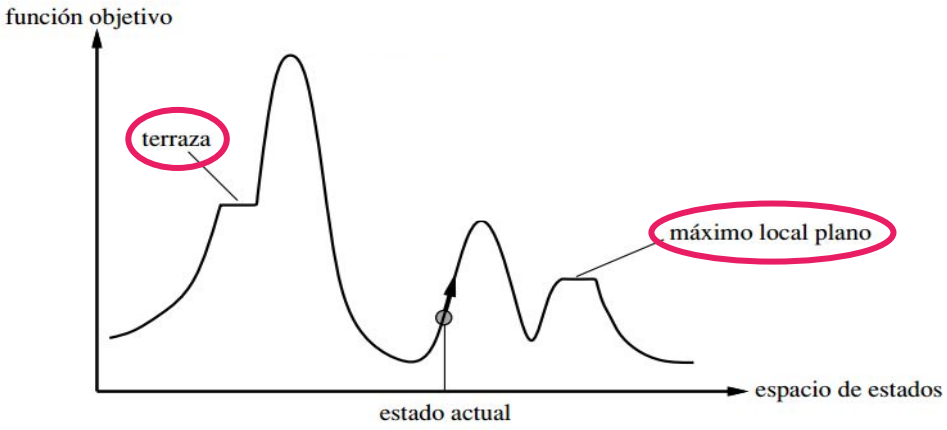
Mesetas

Una **meseta** es un área del paisaje del espacio de estados donde la elevación es plana. Es decir, donde **el mejor estado sucesor tiene el mismo valor objetivo que el estado actual**.

Si no existe una salida ascendente, es un **máximo local plano**, de lo contrario es una **terraza**.



Mesetas



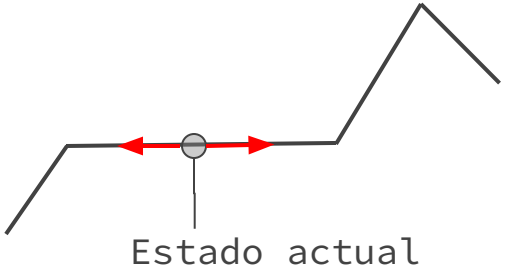
La ascensión de colinas también se **atasca al alcanzar una meseta.**

Tiene sentido retornar si estamos en un máximo local plano (es imposible seguir ascendiendo).

¿Pero si en realidad estamos en una terraza? 🤔

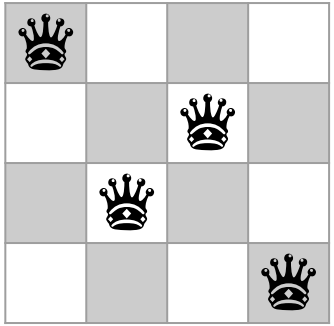
Movimientos laterales

- Es posible moverse por una meseta mediante **movimientos laterales**, que consisten en moverse a un estado sucesor con el mismo valor objetivo que el estado actual.



- **Permitir movimientos laterales en la ascensión de colinas puede permitir escapar de una meseta**.

📖 Ejemplo – 4 Reinas



f(1,3,2,4) = 2

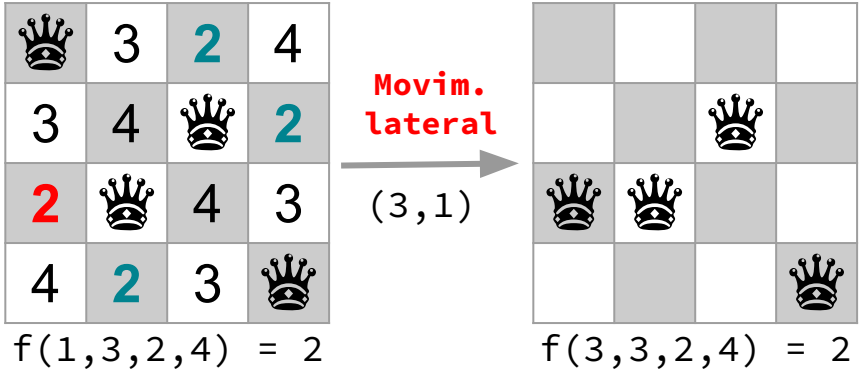
📖 Ejemplo – 4 Reinas

🙁	3	2	4
3	4	🙁	2
2	🙁	4	3
4	2	3	🙁

f(1,3,2,4) = 2

El valor objetivo del mejor sucesor coincide con el valor objetivo actual, es decir, estamos en una **meseta**.
Para intentar desatascar la búsqueda, hacemos un **movimiento lateral**.
Por ejemplo, aplicamos la acción (3,1).

📖 Ejemplo – 4 Reinas



📖 Ejemplo – 4 Reinas

👸	3	2	4
3	4	👸	2
2	👸	4	3
4	2	3	👸

f(1,3,2,4) = 2

Movim. lateral

(3,1)

2	1	2	4
3	2	👸	3
👸	👸	4	4
4	2	3	👸

f(3,3,2,4) = 2

📖 Ejemplo – 4 Reinas

🙁	3	2	4
3	4	🙁	2
2	🙁	4	3
4	2	3	🙁

f(1,3,2,4) = 2

Movim. lateral

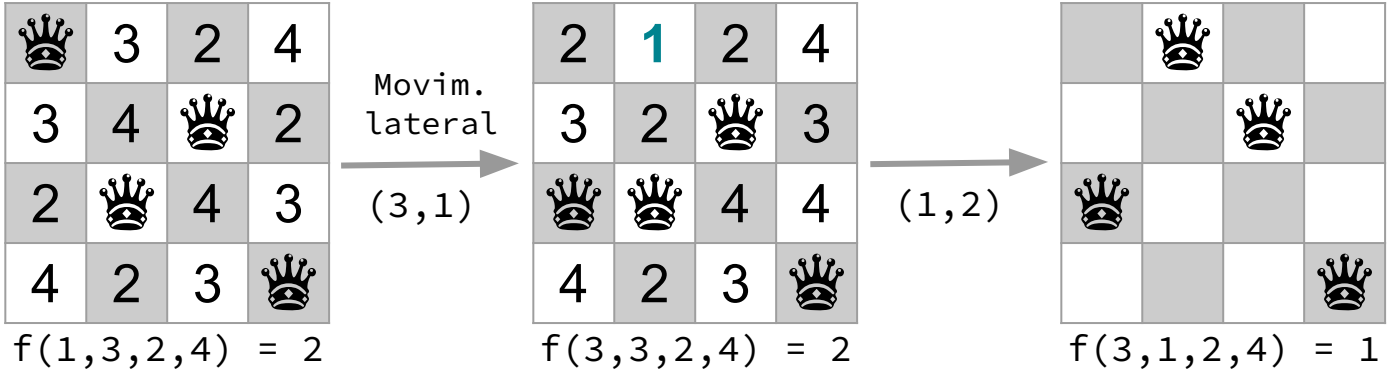
(3,1)

2	1	2	4
3	2	🙁	3
🙁	🙁	4	4
4	2	3	🙁

f(3,3,2,4) = 2

El valor objetivo del mejor sucesor es menor al valor objetivo actual (1 < 2). Es decir, escapamos de la meseta. A continuación, nos movemos a dicho sucesor.

📖 Ejemplo – 4 Reinas



📖 Ejemplo – 4 Reinas

♛	3	2	4
3	4	♛	2
2	♛	4	3
4	2	3	♛

f(1,3,2,4) = 2

Movim. lateral

(3,1)

2	1	2	4
3	2	♛	3
♛	♛	4	4
4	2	3	♛

f(3,3,2,4) = 2

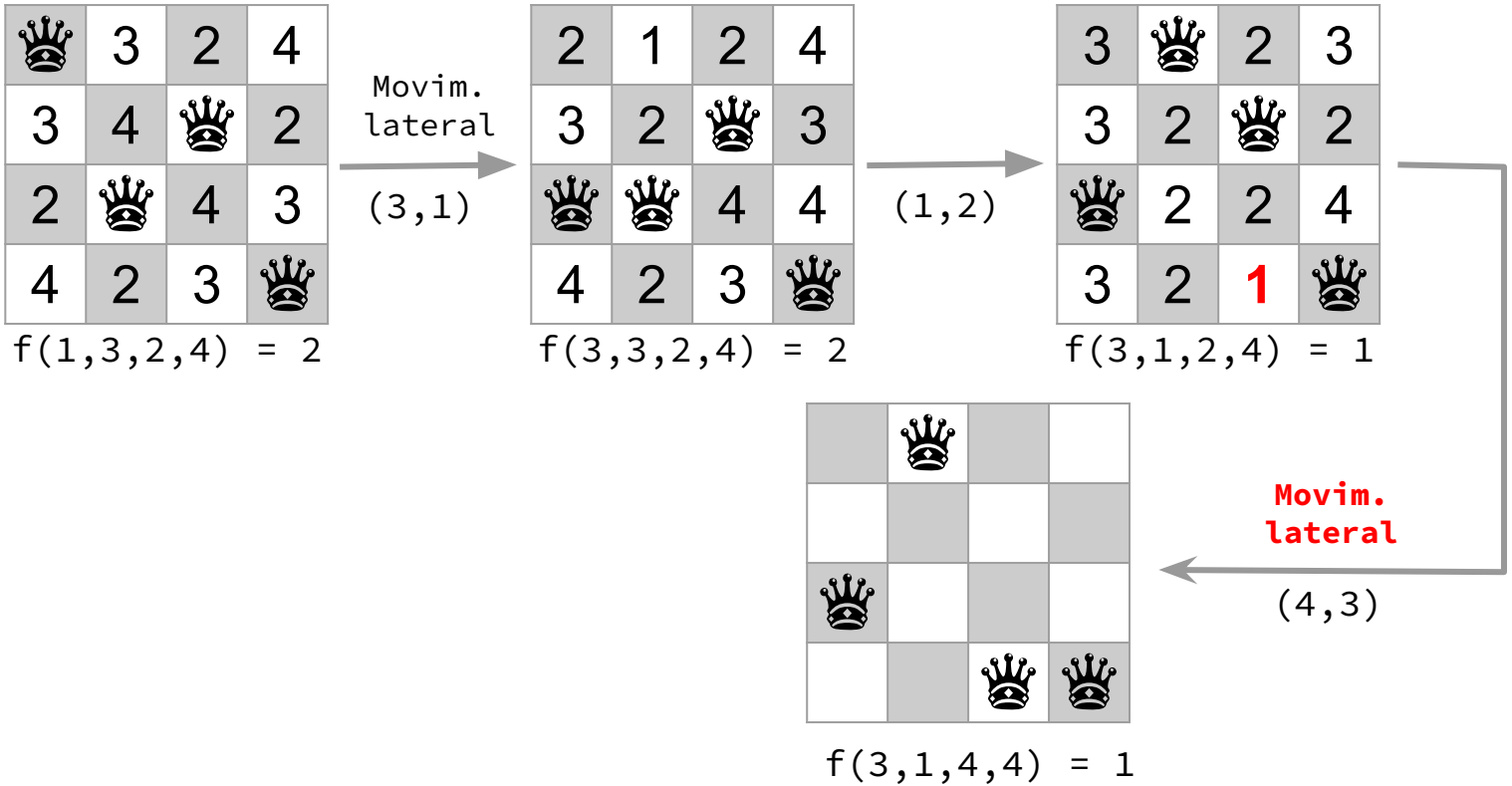
(1,2)

3	♛	2	3
3	2	♛	2
♛	2	2	4
3	2	1	♛

f(3,1,2,4) = 1

El valor objetivo del mejor sucesor coincide con el valor objetivo actual, es decir, estamos nuevamente en una **meseta**. Para intentar desatascar la búsqueda, hacemos otro **movimiento lateral**. Por ejemplo, aplicamos la acción (4,3).

📖 Ejemplo – 4 Reinas



📖 Ejemplo – 4 Reinas

🙁	3	2	4
3	4	🙁	2
2	🙁	4	3
4	2	3	🙁

f(1,3,2,4) = 2

Movim. lateral

(3,1)

2	1	2	4
3	2	🙁	3
🙁	🙁	4	4
4	2	3	🙁

f(3,3,2,4) = 2

(1,2)

3	🙁	2	3
3	2	🙁	2
🙁	2	2	4
3	2	1	🙁

f(3,1,2,4) = 1

Movim. lateral

(4,3)

3	🙁	2	1
3	3	1	0
🙁	3	2	1
3	4	🙁	🙁

f(3,1,4,4) = 1

El valor objetivo del mejor sucesor es menor al valor objetivo actual (0 < 1). Es decir, nuevamente escapamos de la meseta. A continuación, nos movemos a dicho sucesor.

📖 Ejemplo – 4 Reinas

👸	3	2	4
3	4	👸	2
2	👸	4	3
4	2	3	👸

f(1,3,2,4) = 2

Movim. lateral

(3,1)

2	1	2	4
3	2	👸	3
👸	👸	4	4
4	2	3	👸

f(3,3,2,4) = 2

(1,2)

3	👸	2	3
3	2	👸	2
👸	2	2	4
3	2	1	👸

f(3,1,2,4) = 1

Movim. lateral

(4,3)

3	👸	2	1
3	3	1	0
👸	3	2	1
3	4	👸	👸

f(3,1,4,4) = 1

(2,4)

	👸		
			👸
👸			
		👸	

f(3,1,4,2) = 0

Alcanzamos un mínimo global. ¿Siempre ocurre? 🤔

Limitando movimientos laterales

- En el ejemplo anterior, necesitamos un único movimiento lateral para escapar de ambas meseta. No siempre es así…
- Podrían requerirse **múltiples** movimientos laterales consecutivos para escapar. Pero…
- **Permitir infinitos movimientos laterales consecutivos no es garantía de escape.**
  - Podríamos estar en un **máximo local plano** (y no en una terraza).
  - Podríamos estar **ciclando** entre estados con mismo valor objetivo.

  En ambos casos, la ascensión de colinas **no terminaría jamás**.
- Para evitarlo, se suele **limitar** el número de movimientos laterales consecutivos.

💻 Algoritmo de ascensión de colinas con movimientos laterales

---

📝 **Ejercicio.** Escribir el pseudocódigo del algoritmo ASCENSIÓN-COLINA-LATERAL de ascensión de colinas con movimientos laterales (limitando el número de movimientos laterales consecutivos por medio de un parámetro extra).

Características de ASCENSIÓN-COLINA-LATERAL

- Muy bajo consumo de **memoria**.
- Se mueve siempre a un sucesor con valor objetivo **igual o mayor**.
- Se debe **limitar** el número de movimientos laterales consecutivos para garantizar **terminación**.
- Se atasca en un **máximo** (local, global o plano) y a veces también en una **terraza** (cuando no encuentra un escape con el número de movimientos laterales considerados).
- En general, no hay garantía de que el estado retornado sea objetivo (**completitud**), ni que el máximo alcanzado sea global (**optimalidad**).
- Típicamente se atasca con menor frecuencia en un máximo local (respecto a la ascensión de colinas clásica), pero suele necesitar más pasos para terminar. Veamos un ejemplo…

📖 Ejemplo – 8 Reinas

Ascensión de colinas sin movimientos laterales:

- Probabilidad de atascarse en un mínimo local: **86%**.
  Promedio de pasos: **3**.
- Probabilidad de atascarse en un mínimo global: **14%**.
  Promedio de pasos: **4** pasos.

Limitando a 100 el número de **movimientos laterales** consecutivos:

- Probabilidad de atascarse en un mínimo local: **6%**.
  Promedio de pasos: **64**.
- Probabilidad de atascarse en un mínimo global: **94%**.
  Promedio de pasos: **21** pasos.

Otras variantes...

1. **Ascensión de colinas estocástica**.

   Elige el sucesor de forma **aleatoria**, entre todos aquellos que mejoren el valor objetivo actual. También pueden asignarse probabilidades diferenciadas.

   Su convergencia suele ser más lenta, pero a veces encuentra mejores soluciones.

2. **Ascensión de colinas de primera opción.**

   Los estados sucesores se generan de a uno y se elige el **primer** sucesor encontrado cuyo valor objetivo es mejor que el actual.

   Es útil cuando los estados tienen muchos sucesores.

Otras variantes...

3. **Ascensión de colinas con reinicio aleatorio**.

   Realiza una serie de ascensiones de colinas. Cuando se atasca, se **reinicia** desde un estado inicial aleatorio.

   Retorna el estado con mejor valor objetivo encontrado entre todos los reinicios realizados.

Optimalidad. ✅ (con una cierta probabilidad).

Si cada búsqueda de ascensión de colinas tiene probabilidad p de encontrar un máximo global, entonces se esperan $1/p$ reinicios para encontrar un máximo global con probabilidad cercana a **1**.

📖 Ejemplo – 8 Reinas

- Sin movimientos laterales, $p \approx 0.14$, se esperan

  $1/0.14 \approx$ **7.14** reinicios.

- Con movimientos laterales, $p \approx 0.94$, se esperan

  $1/0.94 \approx$ **1.06** reinicios.

El reinicio aleatorio permite resolver problemas con hasta 3M de reinas en menos de un minuto.

📚 Resumen

- Una búsqueda local actúa evaluando y modificando un estado actual, en lugar de explorar sistemáticamente caminos desde el estado inicial.
- Se caracterizan por su bajo consumo de memoria.
- Suelen aplicarse en problemas donde lo único que importa es el estado objetivo, no el camino para alcanzarlo, y típicamente en problemas de optimización.
- Requieren de una formulación de estados completa del problema y de una función objetivo que alcance su máximo valor en estados objetivos.

📚 Resumen

- La búsqueda local más simple es el algoritmo de ascensión de colinas, el cual siempre se mueve a un estado sucesor que mejore el valor objetivo actual.
- El éxito de la ascensión de colinas depende fuertemente del paisaje de espacio de estados. En general, puede quedar atascado en un máximo local que no es global.
- Los movimientos laterales pueden ayudar a desatascar la búsqueda.
- El reinicio aleatorio agrega optimalidad con una cierta probabilidad.

➡️ Próximamente

La ascensión de colinas con pasos laterales **no puede escapar de un máximo local que no es global** si en el paisaje de espacio de estados se encuentra en:

1. Un pico.
2. Un máximo local plano.
3. Una terraza y los movimientos laterales generan un camino cíclico o el número de movimientos laterales no es suficiente para escapar.

¿Cómo escapar en estas condiciones? 🤔